

Deep Dive: \$MFT, Deleted Files & Timestamps

The upstream USB image, solved together — the two new challenges stay yours.

Stefanus Eko Prasetyo

Lab-6 · companion to the Lab 06 lecture

What we cover today

1 Inside an AD1

what the upstream image contains, and how to walk it in FTK

2 \$MFT in depth

record layout, attribute types, parent refs, record numbers

3 MFTECmd workflows

MFT to CSV, boot parsing, zone-identifier exports

4 Timestomping detection

SI vs FN comparison - the two-clock method

5 Worked case + your turn

all five upstream answers, then Challenges B and C

What lives inside Image.ad1

PS C:\Tools>

```
# FTK: Add Evidence Item > Image File > Image.ad1  
root of the imaged USB drive:
```

```
note.txt          33 B   (Q1, Q2)
```

```
mememe.jpeg      49,160 B (Q4 - has an ADS!)
```

```
$MFT $MFTMirr $LogFile $Boot (Q2, Q3, Q5)
```

```
$Txf, $TxfLog, System Volume Information, ...
```

```
# export what you need; the metadata files are
```

```
# the path to every answer
```

Export discipline

- ▶ right-click > Export Files for each artifact
- ▶ hash every export immediately
- ▶ \$MFT export feeds MFTECmd
- ▶ \$Boot export answers Q5 raw

\$MFT record - the hex that matters

PS C:\Tools>

```

0000 46 49 4C 45 30 ...      'FILE0' magic
0016 01 00                  flags: 01=in-use 02=dir
0018 00 04 00 00          record size 1024
0038 30 00 00 00          attribute: 0x30 $FILE_NAME
      +0x00 parent record ref (6 bytes + seq)
      +0x40 name length; +0x42 UTF-16 name
.... 80 00 00 00          attribute: 0x80 $DATA

# record number = offset / 1024

```

Reading recipe

- ▶ magic FILE0 at record start
- ▶ walk attributes: 0x10, 0x30, 0x80
- ▶ FN parent ref rebuilds the tree
- ▶ resident 0x80 = content in-record

MFTECmd - the metadata factory



```
PS C:\Tools> MFTECmd.exe -f "$MFT" --csv . --csvf mft.csv
-> one row per record: rec#, name, parent, SI+FN times, size

PS> MFTECmd.exe -f "$Boot" --csv . --csvf boot.csv
-> volume serial, geometry, OEM name (Q5)

# find a file's record:
PS> Import-Csv mft.csv | ? Name -eq 'note.txt'
EntryNumber : 40      ParentEntry : 5
```

PS C:\Tools>

The two-clock method

Clock	Attribute	Who writes it	Fakable?
Clock 1	\$STANDARD_INFORMATION	user-space APIs (what Explorer shows)	trivially (SetFileTime)
Clock 2	\$FILE_NAME	NTFS kernel, inside the parent dir record	needs raw MFT edits

Method: for every interesting file, put SI and FN times side by side. Agreement = boring. Wild disagreement = timestomping until proven otherwise. Also compare against \$LogFile and neighboring files - a '2019' file in a folder created in 2026 is a story.

Hashes, record numbers, and \$Txf's parent

```
# Q1: FTK Properties pane on note.txt (33 bytes):  
MD5   : c91e969e9184267c35ddc3ff45f795d3  
SHA1  : c61dce75ba83f186471297e2e0568ddd0cefe022  
  
# Q2: MFTECmd on $MFT -> note.txt record number = 40  
(parent record 5 = the root; records 0-11 are the NTFS system files)  
  
# Q3: $Txf record 31 -> parent ref 27 -> record 27 is...  
$RmMetadata (itself under $Extend, under root)
```

PS C:\Tools>

The tweet's origin and the boot serial



PS C:\Tools>

```
# Q4: meme.jpeg has an ADS in FTK's tree:
meme.jpeg:Zone.Identifier -> Export Files
HostUrl: https://pbs.twimg.com/media/FadAHVAUUAAP2Q
        ?format=jpg&name=small

# Q5: $Boot export -> serial at offset 0x48 (8 bytes):
7F 7A 42 44 B7 42 44 F6
(report as hex string 7F7A4244B74244F6; boot also carries geometry
 512 B/sector, 8 sectors/cluster - the volume's birth certificate)
```

FAT deleted-file recovery - the generic recipe

PS C:\Tools>

```
# 1. open the raw image (Add Evidence Item > Raw)
# 2. walk to the root folder - spot the 0xE5 name
4E 45 57 7E 31 54 58 54    normal entry
E5 45 57 7E 31 54 58 54    deleted (first byte E5)
# 3. first-cluster field is zeroed -> old cluster gone?
#    not necessarily: size survives, and content often
#    sits in the data area untouched
# 4. preview/carve clusters until the size matches
```

Why it works

- ▶ deletion clears the FAT chain
- ▶ but not the data clusters
- ▶ small files stay contiguous
- ▶ size field points you to the end

Reading a mini \$MFT - the generic recipe

PS C:\Tools>

```
# 16 records, 1024 bytes each: number = offset/1024
# per record, extract:
  flags          01 file / 03 directory
  FN name + parent ref  -> rebuild the tree
  SI times vs FN times  -> compare years!
  resident $DATA        -> read the content

# one file's two clocks will disagree badly;
# its resident data tells you why
```

Tools

- ▶ MFTECmd works on raw MFT dumps
- ▶ analyzeMFT (python) as well
- ▶ or a hex editor + the layout card
- ▶ report record numbers as citations

Common mistakes, proven habits

Mistake	Habit instead
Trusting Explorer-style timestamps	compare SI and FN - two clocks catch liars
Ignoring the system files	\$MFT/\$Boot/\$LogFile ARE the evidence, not clutter
Exporting without hashing	every export gets hashed and logged immediately
Stopping at the visible tree	check ADS children and deleted entries everywhere
Reporting tools without verification	hex-verify the bytes a parser reports

What a disk image hands the investigator

Artifact	Question it answers
File hashes (FTK properties)	identity and integrity of any file in the image
\$MFT record + parent chain	who exists, where, and what was deleted
Zone.Identifier ADS	where a file was downloaded from, originally
\$Boot serial + geometry	which physical volume this image came from
SI vs FN timestamps	whether times were forged - and how badly
Unallocated / slack	fragments of files no directory remembers

Two fresh images. Same discipline.

01

Challenge B — 4.2

suspect_card.dd: a shipping office's USB stick as a raw FAT16 image. One root entry begins with 0xE5 and its FAT chain is gone - but the bytes are still in the data area. Recover the deleted plan and take its flag.

02

Challenge C — 4.3

mini_mft.bin: sixteen crafted \$MFT records. Rebuild the directory tree from parent refs, compare the two clocks on every file, find the record whose times disagree - and read its resident data.

03

Working paper

pre-lab first; staged hints in section 4; cite record numbers, clusters, and offsets; hash every export.